

The University of Azad Jammu and Kashmir, Muzaffarabad

Name: Muneeba Arshad

Roll NO: 2024-SE-06

Semester:5th

Course Title: Software Construction And Development

Session: 2024-2028

Submitted To: Engr. Asma Javaid

Department of Software Engineering

✓ Lab Report

1. Laboratory Overview

This practical explores how UML can describe the behaviour of systems over time. The activity model is used to map a food-delivery order across the customer, platform, restaurant and rider. The two state-machine models then show how an electric toaster and a passenger elevator respond to user inputs, sensor readings and automatic conditions.

1.1 Learning Outcomes

- Create an activity diagram divided into swimlanes to show which participant performs each step.
- Use fork and join nodes to represent work that happens concurrently and the point at which the work synchronizes.
- Describe toaster states using entry, do and exit behaviours.
- Identify transitions caused by external events and transitions that occur automatically when a condition is met.
- Model elevator travel, door operation and passenger-protection conditions.

1.2 Selecting the Appropriate UML Model

An activity diagram presents a workflow: it follows actions from start to finish and can show decisions, parallel branches and responsibility boundaries. A state machine diagram instead follows the lifecycle of one object or controller. It identifies the states that object can occupy and the events or conditions that cause it to move between states.

2. Task 1 — Food Delivery Application Activity Diagram

2.1 Workflow Description

A food-delivery application connects a person placing an order with a restaurant and a delivery rider. The platform coordinates the transaction and keeps the participants informed. The process begins when

a customer selects food and supplies delivery details. It ends when the order reaches the customer and feedback can be submitted.

2.2 Swimlane Responsibilities

Participant	Main responsibility	Typical actions
Customer	Starts and completes an order	Explore restaurants, select dishes, provide delivery details, choose payment and rate the order.
Food-delivery platform	Coordinates the transaction	Validate payment, create the order record, forward order details, find a rider and update tracking.
Restaurant / kitchen	Prepares the meal	Review the request, accept or decline it, cook and pack the food, then mark it ready.
Delivery rider	Moves the package to the customer	Accept dispatch, reach the restaurant, collect the order and deliver it to the given address.

2.3 Decisions and Parallel Activities

Payment choice: The customer can pay electronically or select cash on delivery. If an online transaction is declined, the customer is asked to try again. Once payment is accepted—or COD is selected—the platform can proceed with order registration.

Restaurant decision: The kitchen checks whether it can fulfil the request. If it declines the order, the platform cancels the transaction and starts the applicable refund process.

Parallel work: After acceptance, meal preparation and rider assignment can proceed at the same time. This is represented by a fork in the activity flow.

Synchronization: The rider should receive the package only after the meal is ready and the rider has reached the restaurant. A join represents the point where both branches have completed.

2.4 Activity Diagram

The four vertical partitions show where each action belongs. The payment route has two alternatives: online payment or cash on delivery. A declined online payment can be retried. After the restaurant

accepts the request, food preparation and rider assignment proceed in parallel. The flow joins when both the prepared package and the rider are available for handover.

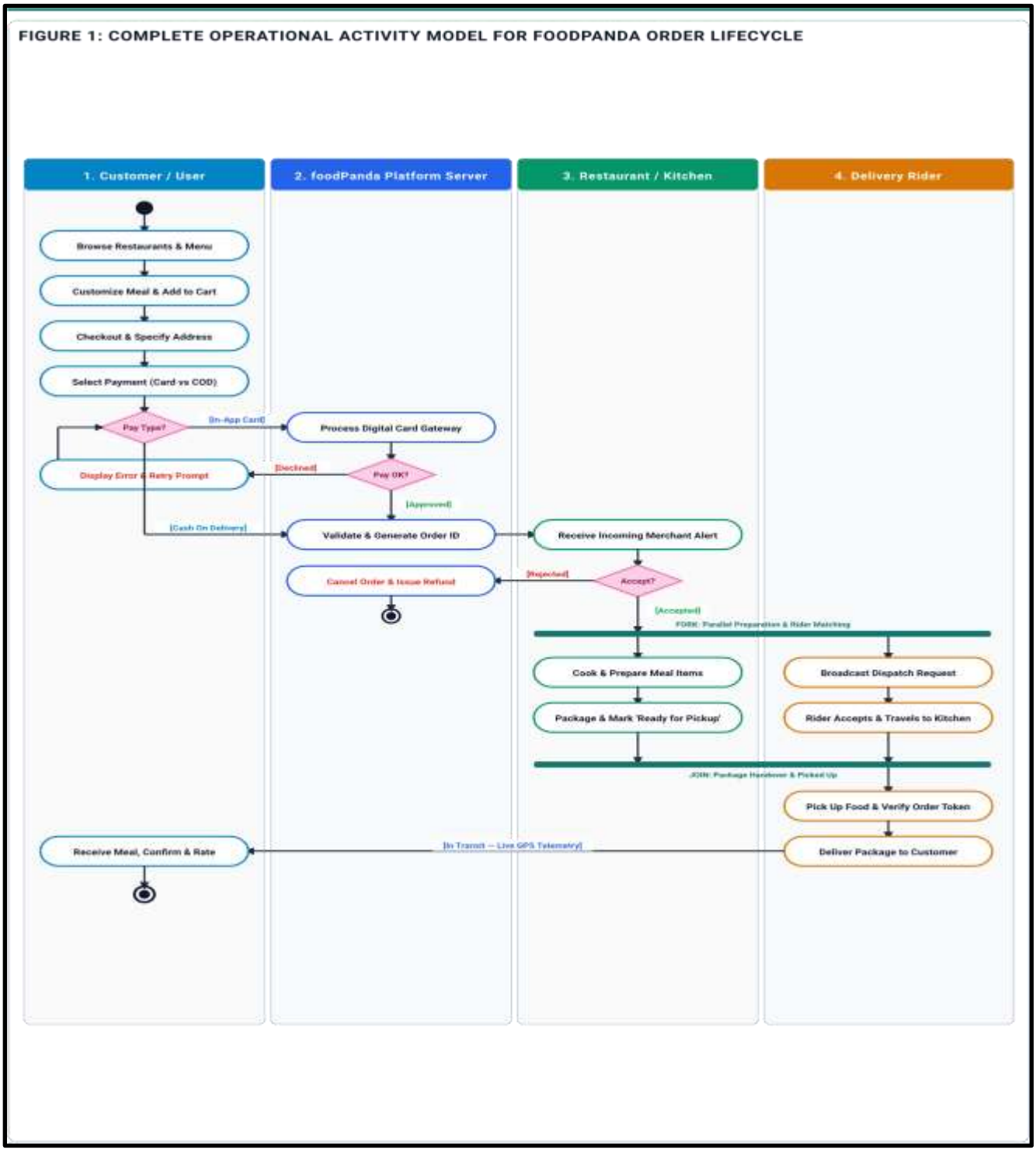


Figure 1. Activity model of an online food order and delivery workflow.

3. Task 2 — Electric Toaster State Machine

3.1 System Description

An electric toaster moves through a sequence of operating conditions. It waits for bread, detects that bread has been inserted, heats the bread according to the selected setting, and then switches off the heater before raising the toast. Temperature protection is included so that heating can be interrupted if an unsafe condition is detected.

3.2 State Activities

The entry behaviour runs when a state is entered, the do behaviour continues while the toaster remains in that state, and the exit behaviour is performed as the toaster leaves it.

State	Entry behaviour	Activity while in state	Exit behaviour
Idle / Ready	Turn on ready indicator and enable slot sensing.	Check bread slots and read the browning control.	Prepare the latch mechanism.
Bread Loaded	Recognize bread and provide a small confirmation sound.	Read the selected browning level and wait for lever operation.	Set the heating profile.
Toasting	Engage the latch, power the heating coils and start the timer.	Control temperature, update the timer and monitor the chamber.	Switch off the heating coils and record the cycle.
Cooling / Ejecting	Release the latch, raise the toast and sound the alert.	Allow residual heat to dissipate and wait for bread removal.	Reset the sensors and timer.
Overheat Fault	Disconnect heater power and indicate a fault.	Keep the carriage locked while chamber temperature falls.	Reset the thermal cut-off after safe conditions return.

3.3 Triggered and Triggerless Transitions

A triggered transition depends on an event such as a button press or bread insertion. A triggerless transition occurs automatically when an internal condition or completion condition becomes true.

Transition type	Event / condition	From → To	Resulting action
Triggered	insertBread()	Idle → Bread Loaded	Activate the bread-presence sensor.
Triggered	pushLeverDown() [powerOK]	Bread Loaded → Toasting	Latch the carriage and start the heating timer.
Triggered	pressCancelButton()	Toasting → Cooling / Ejecting	Stop the cycle and remove heater power.
Triggerless	timerExpired OR browningAchieved	Toasting → Cooling / Ejecting	Turn off the coils automatically.
Triggerless	toastRemoved AND	Cooling / Ejecting → Idle	Restore the ready condition.

	tempNormalized		

3.4 State Machine Diagram

The model begins in the ready condition. Bread insertion leads to the loaded state, and lowering the lever starts the heating cycle when power is available. The timer or selected browning level can complete the heating cycle automatically. The cancel control provides an alternative route to cooling. An overheat condition moves the toaster into its fault-protection state.

z

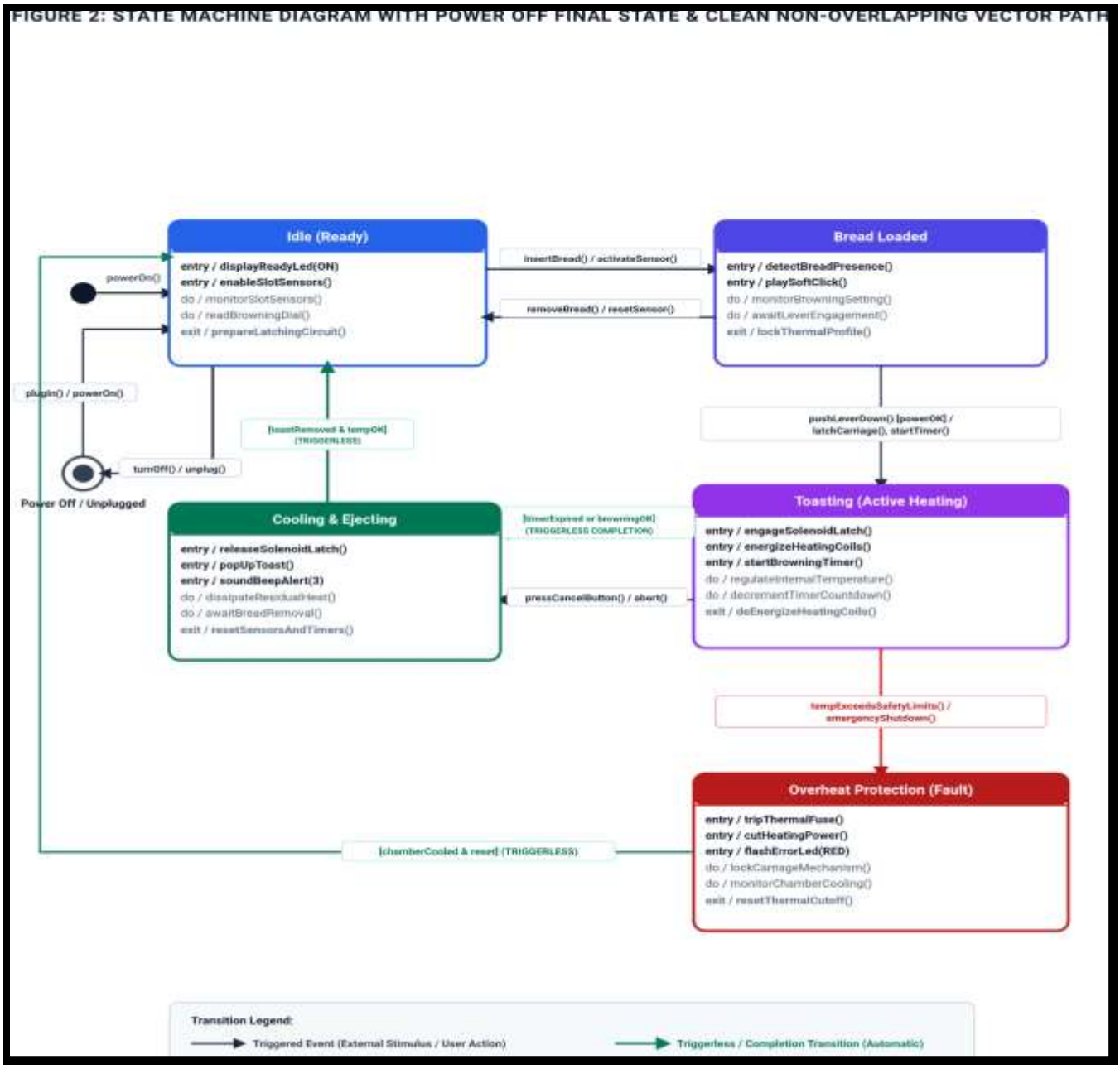


Figure 2. State machine for an electric toaster, including normal and fault behaviour.

4. Task 3 — Passenger Elevator State Machine

4.1 System Description

A passenger elevator controller receives floor requests, selects a direction, controls the motor and brake, and manages the doors at each stop. Sensors provide information about floor alignment, door position, obstructions and cabin load. The controller must check these conditions before allowing another movement cycle.

4.2 State Transition Table

Current state	Event	Guard / condition	Action	Next state
Idle at floor	callRegistered	Requested floor is higher	Release brake and command upward travel	Moving Up
Idle at floor	callRegistered	Requested floor is lower	Release brake and command downward travel	Moving Down
Moving	targetFloorApproached	Level sensor is active	Reduce speed and apply brake	Idle at floor
Idle at floor	arrivedAtFloor	Cabin is stopped and aligned	Open the door mechanism	Doors Opening
Doors Opening	limitSwitchReached	Door is fully open	Start the passenger waiting interval	Doors Open
Doors Open	dwellTimerExpired	Doorway is clear and load is acceptable	Command door closure	Doors Closing
Doors Open	weightSensorTripped	Load is above rated capacity	Sound warning and keep doors open	Overload Warning

4.3 Safety Conditions

Door interlock: The lift must not begin moving until the car-door and landing-door locking system confirms that the doors are secured.

Obstacle response: If the doorway sensor detects an obstruction during closure, the controller should stop closing and reopen the doors.

Overload handling: When the measured load exceeds the permitted capacity, travel remains suspended and the doors stay open until the load is within the limit.

Emergency condition: An emergency input or loss of power causes the controller to enter its emergency response condition and apply the designated braking and safety procedures.

4.4 State Machine Diagram

When the elevator is idle, a request for a higher or lower floor selects the corresponding direction of travel. Once the target floor is reached, the controller stops the cabin and opens the doors. Passengers are given a short boarding interval before closure begins. An excessive load keeps the doors open and suspends travel. Emergency input or power loss is represented as a separate safety condition.

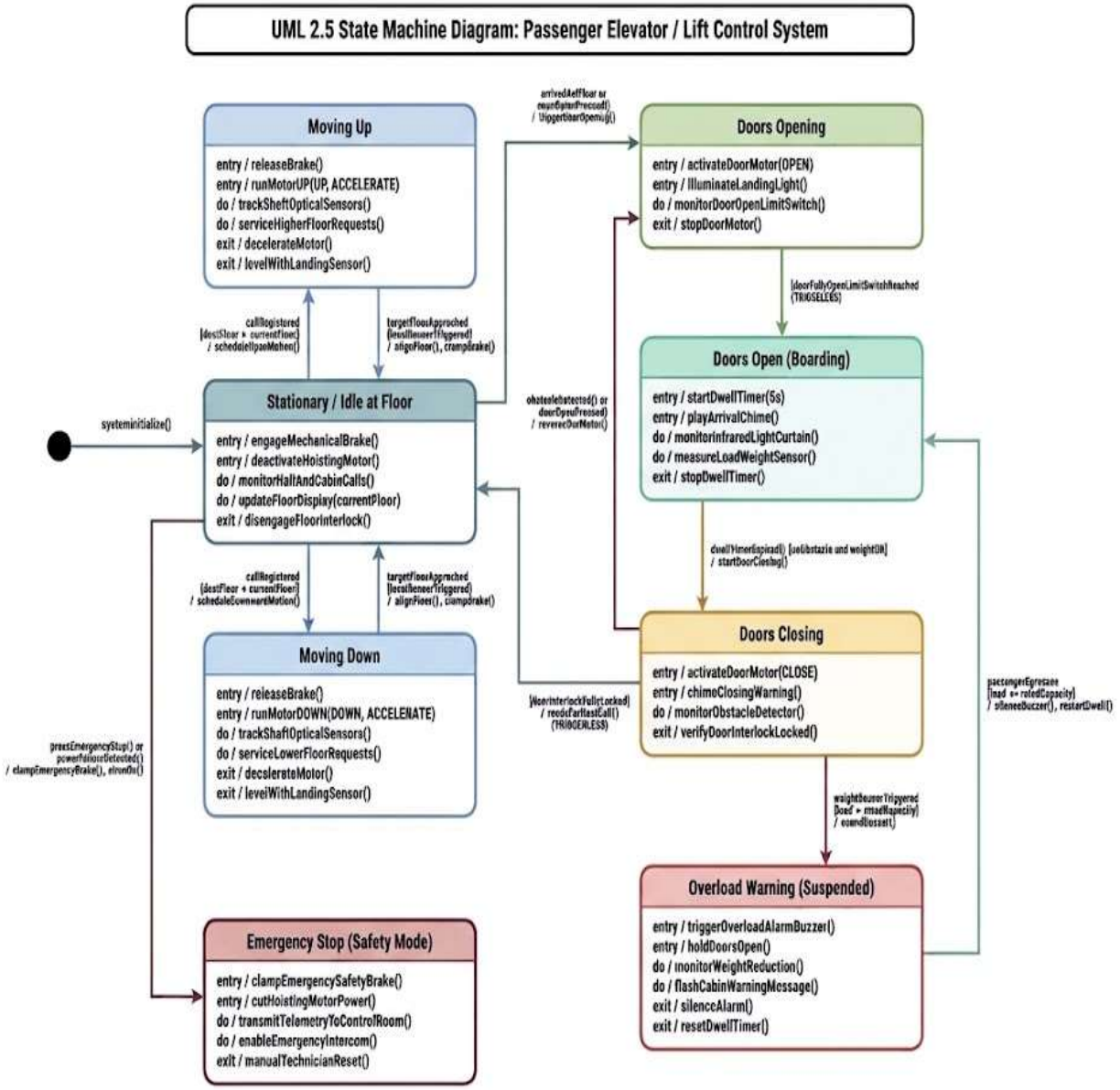


Figure 3. Elevator state model showing travel, door handling and safety responses.

5. Comparison of the Two Behavioural Models

Both diagram types describe behaviour, but they answer different modelling questions. The activity model follows the work performed by several participants, whereas a state machine follows the changing condition of one system or object.

Aspect	Activity diagram	State machine diagram
Main focus	Sequence of work across participants or components.	Operating conditions of one system or object.
Basic element	Action or activity node.	State.
Flow control	Decisions, forks and joins.	Events, guards and state transitions.
Common use	Business processes and coordinated services.	Appliances, controllers and event-driven systems.

6. Conclusion

The three models demonstrate how UML behavioural diagrams can describe both coordinated workflows and event-driven devices. The food-delivery activity diagram assigns work to the customer, platform, restaurant and rider, while its fork and join show parallel preparation and dispatch. The toaster model records its operating states, internal activities and automatic or user-triggered transitions. The elevator model represents floor movement and door control together with overload, obstruction and emergency responses. These diagrams provide a structured description of system behaviour that can support later design and implementation.